Algorithm Design and Loop Invariants

Lecture 32

CS 1001 Kirk Duran

Today

- Explain why a loop works by describing the meaning of its state.
- Design Two Sum using pairs, then remembered information.
- Check nondecreasing order using neighboring elements.
- Traverse nested objects and group students by major.
- Track a current positive streak and the best streak seen so far.
- Carry the same reasoning into Lab 6: strings and mixed data.

Key idea

What must the program remember, and what should that state mean after each iteration?

From Last Time: A Sorted Region Means Something

Selection sort

The finished prefix contains the smallest values in their final sorted positions.

Insertion sort

The processed prefix contains its original values, now in sorted order.

- We used these statements to explain each algorithm's progress.
- The statement stays true as the relevant region grows.
- Today we will name this reasoning and use it to design new loops.

Our Design Routine

Prompt $\longrightarrow$ **Contract** $\longrightarrow$ **Examples**

$\longrightarrow$ **State and invariant** $\longrightarrow$ **Code** $\longrightarrow$ **Tests**

- Decide what the function receives, returns, and may change.
- Work out ordinary and boundary examples before coding.
- Give each state variable a precise job.
- Explain why one iteration preserves those meanings.

Key idea

Choose the state from the problem, rather than choosing variable names first and hoping they work.

Part 1: What Makes a Loop Correct?

Definition: Loop Invariant

Definition

A **loop invariant** is a statement about program state that holds every time execution reaches a specified point in a loop.

Our usual checkpoint

Immediately before the next iteration: what is already true about the portion processed so far?

- State the checkpoint; a claim need not hold halfway through an update.
- The variables may change while the relationship remains true.
- A useful invariant connects the changing state to the required result.

A Familiar Invariant: Counting Matches

Count values equal to a target

```
def count_matches(values: list[int], target: int) -> int:
    count = 0
    for value in values:
        if value == target:
            count += 1
    return count
```

Key idea

Before each iteration, `count` equals the number of target matches in the values already processed.

In class

What does the claim say before the first iteration? After the last?

Three Questions About the Invariant

1. **Initialization:** Why is it true before any iterations?
   No values have been processed, so the matching count is 0.
2. **Maintenance:** Why does one iteration preserve it?
   A match adds one; a nonmatch adds nothing. The count now includes the new value.
3. **Conclusion at exit:** What follows when processing finishes?
   Every value has been processed, so the count describes the entire input.

Key idea

An invariant explains a whole family of intermediate states, rather than only one example run.

Correct State and Eventual Termination

Does the state stay meaningful?

Use initialization and maintenance. Combine the invariant with the exit condition to explain the answer.

Will execution reach the exit?

A `for` loop over our finite, unchanged input eventually exhausts it. A `while` loop needs progress toward a false condition.

- For binary search, the remaining interval strictly shrinks.
- For an early return, explain why the answer is justified at that point.
- Passing tests gives evidence; invariant reasoning explains why the updates work generally.

Part 2: Two Sum

Prompt 1: Find Two Values with a Target Sum

In class

Given a list of integers and a target, return two distinct indexes whose values add to the target. Return `None` if no such pair exists.

Values	Target	One valid result
`[2, 7, 11, 15]`	9	`(0, 1)`
`[3, 3]`	6	`(0, 1)`
`[3]`	6	`None`

Contract

Return indexes, not values. Any valid pair is acceptable. Do not modify the input.

Two Sum: Design Before Code

In class

How can we examine every pair without pairing an element with itself?

1. Choose a first index `i`.
2. Choose a second index `j` strictly after `i`.
3. Check whether their values add to the target.
4. Return on a match; report failure only after all pairs are ruled out.

Key idea

Using `j > i` guarantees distinct positions and avoids examining the same pair in both orders.

Two Sum: Examine Pairs Directly

First correct solution

```
def two_sum_pairs(values: list[int], target: int):
    for i in range(len(values)):
        for j in range(i + 1, len(values)):
            if values[i] + values[j] == target:
                return (i, j)
    return None
```

- Before each outer iteration, every pair with a first index less than `i` has been checked without a match.
- The inner loop examines every possible later partner for `i`.
- In the worst case there are $n(n-1)/2$ pairs: quadratic growth.

Two Sum: What Information Would Save Work?

Suppose the target is 9 and the current value is 7

The missing value is `9 - 7`, which is 2.

In class

Can we remember whether 2 occurred earlier, along with where it occurred?

- A dictionary can map a previously seen **value** to an earlier **index**.
- For each current value, look for `target - value`.
- If it exists, the earlier index and current index form a pair.
- Otherwise, remember the current value for later iterations.

Key idea

The dictionary stores exactly the information a future iteration needs.

Two Sum: Remember Earlier Values

Dictionary-based solution

```
def two_sum(values: list[int], target: int):
    seen = {}
    for i, value in enumerate(values):
        needed = target - value
        if needed in seen:
            return (seen[needed], i)
        seen[value] = i
    return None
```

Key idea

Before index `i` is processed, `seen` records an earlier index for every distinct value in `values[:i]`.

Trace Two Sum: A Match Can Use Equal Values

For `values = [3, 3]` and `target = 6`:

i	value	needed	seen before check	Action
0	3	3	`{}`	store `3: 0`
1	3	3	`{3: 0}`	return `(0, 1)`

In class

What happens for `[3]` with the same target? What if we store the current value before checking?

Key idea

Check first, then store. Earlier indexes are distinct from the current index; equal values are still allowed.

Why the Dictionary Solution Works

1. **Initialize:** the processed prefix is empty and `seen` is empty.
2. **Maintain:** after an unsuccessful check, record the current value's index. Every distinct processed value still has a representative.
3. **Successful exit:** the saved index is earlier, and its value plus the current value equals the target.
4. **Exhaustion:** if a valid pair existed, its later member would have found the earlier member's value. No pair was missed.

Work and storage

One traversal; under the usual average constant-time dictionary-operation model, expected $O(n)$ time and up to $O(n)$ extra storage. The pair solution uses $O(1)$ extra storage.

Two Sum: Predict and Test

In class

Predict the dictionary solution's return value for each case.

Values	Target	What this checks
`[]`	4	no iterations
`[4]`	8	cannot reuse one position
`[4, 4]`	8	repeated values, distinct positions
`[-3, 8, 2]`	5	negative values
`[1, 2, 4]`	8	no pair
`[1, 4, 2, 3]`	5	multiple possible answers

Key idea

A general correctness test checks the returned indexes and their sum; different correct algorithms may choose different pairs.

Part 3: Neighboring Values

Prompt 2: Is the List Nondecreasing?

In class

Return `True` exactly when each value is at least as large as its predecessor. Leave the list unchanged.

Input	Result
`[2, 2, 5, 9]`	`True`
`[2, 5, 3, 9]`	`False`
`[]`	`True`
`[7]`	`True`

Contract detail

Equal neighbors are allowed. Empty and singleton lists have no violating adjacent pair.

Nondecreasing: Choose the Comparisons

```
[2, 2, 5, 9]

2 <= 2    2 <= 5    5 <= 9
```

In class

Which indexes identify these pairs? Why should the current index start at 1?

- Compare `values[i - 1]` with `values[i]`.
- One decrease is enough to disprove the whole claim.
- One correctly ordered pair is not enough to prove the whole claim.

Key idea

Before checking index `i`, every adjacent pair ending before `i` has passed.

Nondecreasing: Implementation

Look for a counterexample

```
def is_nondecreasing(values: list[int]) -> bool:
    for i in range(1, len(values)):
        if values[i] < values[i - 1]:
            return False
    return True
```

- Before the first check, no adjacent pairs need to have passed yet.
- A successful comparison extends the checked prefix by one value.
- Reaching the final return means every adjacent pair passed.
- For fewer than two values, the loop is empty and returns `True`.

Nondecreasing: Trace and Repair

i	Previous	Current	Action for [2, 5, 3, 9]
1	2	5	no decrease; continue
2	5	3	decrease; return `False`
3	—	—	never reached

In class

What goes wrong if `return True` is inside the loop, immediately after the `if` statement?

Key idea

A successful first comparison does not establish the property for the unchecked suffix.

Work

At most $n - 1$ comparisons for $n \geq 1$: $O(n)$ worst-case time and $O(1)$ extra storage. We do not need to sort the input to check it.

Part 4: Algorithms over Nested Objects

Our Shared Object Model

Object	Attributes
`Course`	`code`, `students: list[Student]`
↓	the roster contains references to student objects
`Student`	`name`, `major`, `scores: list[int]`
↓	scores are in chronological order

Contracts for today's examples

Each student appears once in the roster, names are unique within a course, and every score list is finite. Algorithms read the course and students without changing them.

Key idea

Follow the data one level at a time: course, student, then score.

Define the Objects We Will Use

Student state

```
class Student:
    def __init__(self, name: str,
                 major: str,
                 scores: list[int]):
        self.name = name
        self.major = major
        self.scores = scores
```

A course contains students

```
class Course:
    def __init__(self, code: str,
                 students: list[Student]):
        self.code = code
        self.students = students
```

Key idea

These assignments store references. The constructors do not automatically copy the roster or score lists.

Construct One Course and Three Students

Shared data for both object exercises

```
ada = Student("Ada", "CS", [2, 3, 0, 4, 5, 6, 0])
grace = Student("Grace", "Math", [0, 5, 6])
linus = Student("Linus", "CS", [])
course = Course("CS 1001", [ada, grace, linus])
```

In class

Predict each expression before running it.

```
course.students[0] is ada      ?
course.students[1].major       ?
course.students[0].scores[3]   ?
```

Prompt 3: Group a Course's Students by Major

In class

Given a `Course`, return a new dictionary mapping each major to a new list of that major's students. Preserve roster order within each group.

For our example course

```
{"CS":  [ada, linus], "Math":  [grace]}
```

- The values in each group are the original student objects.
- The dictionary and its group lists are new containers.
- An empty roster produces an empty dictionary.

In class

What does a group store that a frequency counter does not?

Grouping: Plan and Implementation

One group for each observed major

```
def group_by_major(course: Course) -> dict[str, list[Student]]:
    groups = {}
    for student in course.students:
        major = student.major
        if major not in groups:
            groups[major] = []
        groups[major].append(student)
    return groups
```

Key idea

After each iteration, every processed student appears exactly once in the correct group, in roster order.

Start empty; create each bucket once; append each student once.

Grouping: Trace the Changing Containers

Student	Major	groups after processing
—	—	`{}`
Ada	CS	`{"CS": [ada]}`
Grace	Math	`{"CS": [ada], "Math": [grace]}`
Linus	CS	`{"CS": [ada, linus], "Math": [grace]}`

In class

Where is each new list created? Why must `groups[major] = []` run only when the major is first encountered?

Key idea

Reinitializing an existing group's list would discard earlier students from that group.

New Containers Can Still Share Their Elements

Predict each effect

```
groups = group_by_major(course)
print(groups["CS"][0] is course.students[0])
groups["CS"].pop()
print(len(course.students))
groups["CS"][0].name = "Ada Lovelace"
print(course.students[0].name)
```

In class

Which statements change a container? Which change a shared student?

Reference Exercise: Reveal

Output	Reason
`True`	both containers refer to the original Ada object
`3`	removing from a group does not remove from the roster
`Ada Lovelace`	attribute assignment changes the shared student

Key idea

A fresh outer container does not imply fresh copies of the objects inside it.

Reset for the remaining examples

Restore `ada.name = "Ada"`. The grouping function itself left the original objects unchanged; the caller performed these later mutations.

Part 5: Current Streak and Best Streak

Prompt 4: A Student's Longest Positive Streak

In class

Add a method that returns the longest consecutive run of scores greater than zero. Leave the student's scores unchanged.

Scores	Longest positive streak
`[2, 3, 0, 4, 5, 6, 0]`	3
`[0, 5, 6]`	2
`[]`	0
`[-2, 0, -1]`	0

In class

Why does counting all positive scores answer a different question?

Streaks: Give Two Variables Different Jobs

`current`

The number of consecutive positive scores at the **end** of the processed prefix.

`best`

The longest positive streak **anywhere** in the processed prefix.

- A positive score extends `current`; any other score resets it.
- Update `best` if the current streak exceeds it.
- A reset of `current` must not erase `best`.

Key idea

At the iteration boundary, both meanings must hold. Neither variable alone represents both facts.

Streaks: Add a Method to Student

Place this method inside the existing Student class

```
def longest_positive_streak(self) -> int:
    current = 0
    best = 0
    for score in self.scores:
        if score > 0:
            current += 1
        else:
            current = 0
        if current > best:
            best = current
    return best
```

Key idea

Reads `self.scores`; returns a number without mutating the student.

Streaks: Trace Ada's Scores

Score	Action	current after update	best after update
—	initialize	0	0
2	extend	1	1
3	extend	2	2
0	reset	0	2
4	extend	1	2
5	extend	2	2
6	extend	3	3
0	reset	0	3

In class

What would change if the scores ended at 6? Why does returning `current` fail for the full example?

Streaks: Explain Correctness

1. **Initialize:** an empty prefix has an ending streak of length zero and a best streak of length zero.
2. **Maintain current:** a positive score extends the ending streak; a nonpositive score makes its length zero.
3. **Maintain best:** the only new candidate for a longer streak ends at the new score. Compare its length with the previous best.
4. **At exit:** the processed prefix is the full score list, so `best` is the required result.

Termination and work

The method traverses one finite score list: $O(m)$ time for m scores and $O(1)$ extra storage.

Now Report the Streak for Every Student

In class

Return a dictionary mapping each student's unique name to their longest positive streak.

The outer traversal follows the object structure

```
def streak_report(course: Course) -> dict[str, int]:
    report = {}
    for student in course.students:
        report[student.name] = student.longest_positive_streak()
    return report
```

For our example: `{"Ada": 3, "Grace": 2, "Linus": 0}`.

Key idea

The outer loop visits students; each method call traverses that student's scores and starts with fresh local accumulators.

Nested Traversal: State, Work, and Boundary Cases

- **Outer invariant:** the report contains the correct result for every student already processed.
- **Inner invariant:** `current` and `best` describe the processed scores of one student.
- Each method call initializes its own `current` and `best`; one student's streak does not carry into another's.
- An empty course gives `{}`; a student with no scores gets `0`.
- With s students and q total scores, the work is $O(s + q)$; students can have different numbers of scores.

In class

Why would a report keyed by name lose information if two students had the same name?

Streaks: Predict Before Running

Scores	What this checks
`[]`	no iterations
`[5]`	one positive score
`[0, -2]`	no positive score
`[1, 2, 3]`	all positive; best streak at the end
`[1, 2, 0, 3]`	best streak before the end
`[1, 0, 2, 0, 3]`	separated positives
`[1, 2, 0, 3, 4]`	tied longest streaks

In class

Give each expected result. Then choose two students whose scores would expose a failure to reset state between students.

Part 6: Carry the Reasoning into Lab 6

Lab 6: Different Data, Familiar State Meanings

Task	What should be true of the processed prefix?
`count_character`	The count equals the matching characters seen so far.
`swap_case`, `str_translate`	The new string is the required transformation of the processed characters.
`remove_character`, `words_of_length`	The result contains exactly the selected characters or words, in encounter order.
`groups_of_3`	The groups built so far, plus any unfinished group, account for every processed value exactly once, in order.

Key idea

Strings are immutable: construct a new result. For nested lists, give each output group its own container.

Lab 6: Turn the Contract into Tests

- Use `unittest.TestCase` and `self.assertEqual(...)` for your added cases.
- Test empty inputs, one element, first/last matches, no matches, and repeated matches.
- Check that transformations and filters preserve the required order.
- For `groups_of_3`, test lengths divisible by three and final groups of one or two.
- Check that the input list remains unchanged and distinct output groups do not share one mutable list.
- Follow the lab's restrictions on shortcut string methods.

In class

For an eight-element input to `groups_of_3`, what must still be handled after the last full group?

Exit Ticket: Explain the State

1. Why must Two Sum check the dictionary before storing the current value?
2. Why can nondecreasing return `False` early but not `True` after just one successful comparison?
3. Why do `current` and `best` need different meanings?
4. If two output groups share one list object, what happens when one group is appended to?

Key idea

An explanation should connect the update to the meaning of the state, rather than only repeat the Python statement.

Takeaways

1. A loop invariant states what is true at a specified iteration boundary.
2. Explain initialization, maintenance, the conclusion at exit, and termination.
3. A dictionary can remember information that avoids repeated searching.
4. Adjacent comparisons answer questions about neighboring values.
5. Nested objects require following references and restarting local state at the right level.
6. Fresh containers can hold shared objects; distinguish container changes from element changes.
7. Give each accumulator a precise job, then test the boundaries of that meaning.

Next

Input, output, and string processing. In Lab 6, apply today's reasoning to strings and nested lists.